



PIZZA SALES ANALYSIS USING SQL

HELLO

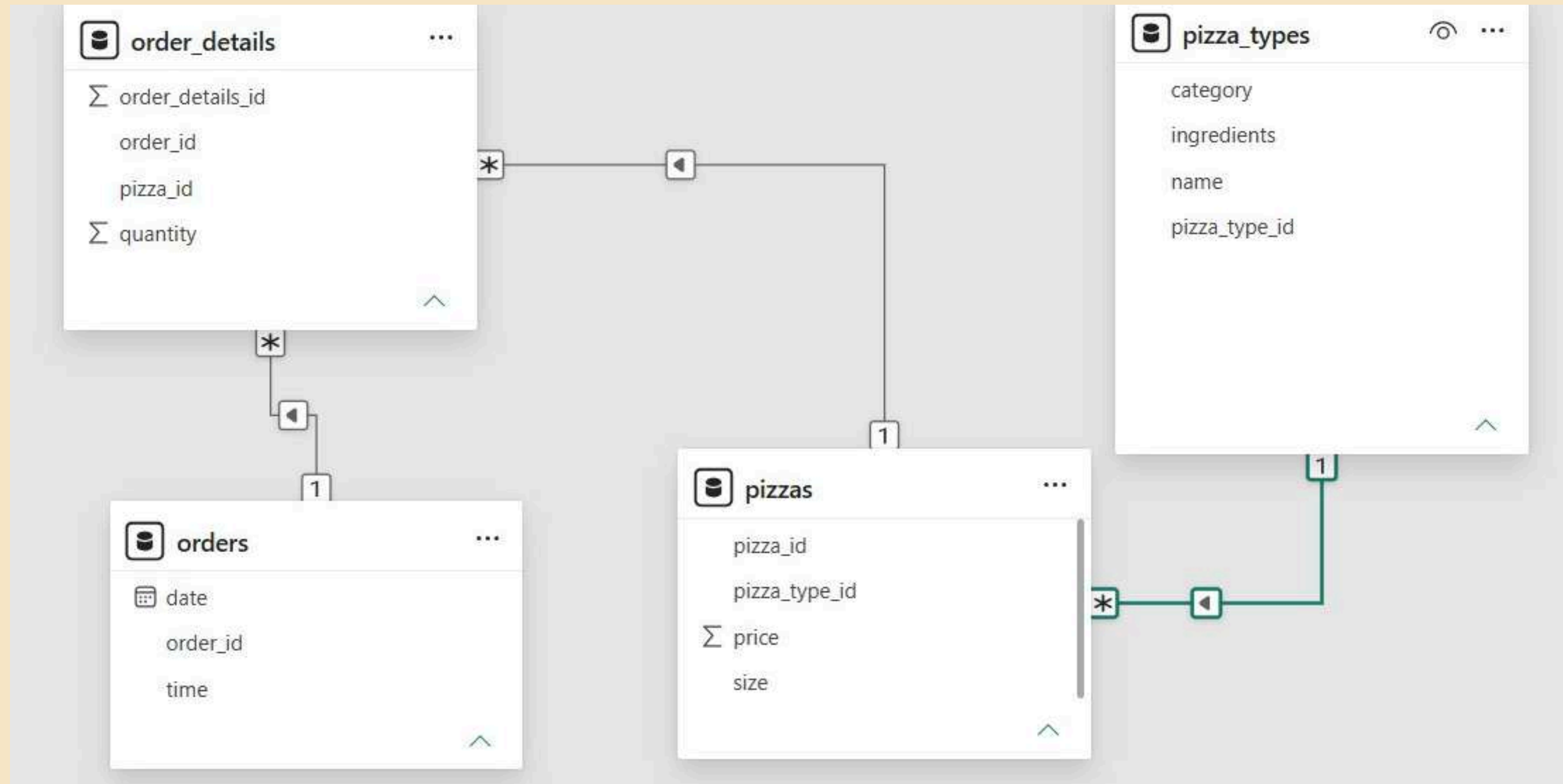
My name is ShahilFarshad,and I ampresenting my project on Pizza Sales Analysis usingSQL.

This project focuseson analyzing pizzasales data using SQL to gain valuable business insights. Byapplying SQL concepts such as joins, aggregate functions, grouping, Subqueries, and window functions, I analyzed sales trends, customer ordering patterns, revenue performance, and product popularity.

The goal of this project is to demonstrate how SQL can transform raw sales data into meaningful insights that support informed business decisions and improve overall business performance.



PIZZA SALES DATABASE SCHEMA



Illustrates the relationships between the Orders, Order Details, Pizzas, Pizza Types, and Categories tables used for SQL analysis.



Retrieve the total number of orders placed.



```
SELECT  
    COUNT(order_id) AS `Total Orders`  
FROM  
    orders;
```

Result Grid	
	Total Orders
▶	21350



Calculate the total revenue generated from pizza sales.



```
SELECT  
    round(sum(order_details.quantity * pizzas.price),2) as `Total Revenue`  
FROM  
    order_details  
    JOIN  
    pizzas ON order_details.pizza_id = pizzas.pizza_id;
```

Result Grid	
	Total Revenue
▶	817860.05



Identify the highest-priced pizza.



```
SELECT
    pizza_types.name, pizzas.price
FROM
    pizza_types
    JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
ORDER BY pizzas.price DESC
LIMIT 1;
```



Result Grid			Filter Rows
	name	price	
▶	The Greek Pizza	35.95	



Identify the most common pizza size ordered.



```
SELECT
    pizzas.size,
    COUNT(order_details.order_details_id) AS order_count
FROM
    pizzas
    JOIN
    order_details ON pizzas.pizza_id = order_details.pizza_id
GROUP BY pizzas.size
ORDER BY order_count DESC
LIMIT 1;
```

Result Grid			Filter
	size	order_count	
▶	L	18526	



List the top 5 most ordered pizza types along with their quantities.



```
SELECT
    pizza_types.name, SUM(order_details.quantity) AS Quantity
FROM
    pizza_types
    JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
    JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.name
ORDER BY Quantity DESC
LIMIT 5;
```

Result Grid			Filter Rows:
	name	Quantity	
▶	The Classic Deluxe Pizza	2453	
	The Barbecue Chicken Pizza	2432	
	The Hawaiian Pizza	2422	
	The Pepperoni Pizza	2418	
	The Thai Chicken Pizza	2371	



Join the necessary tables to find the total quantity of each pizza category ordered.



```
SELECT
    pizza_types.category,
    SUM(order_details.quantity) AS `Total quantity`
FROM
    pizza_types
    JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
    JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.category
ORDER BY `Total quantity` DESC;
```

Result Grid			Filter Rows
	category	Total quantity	
▶	Classic	14888	
	Supreme	11987	
	Veggie	11649	
	Chicken	11050	



Determine the distribution of orders by hour of the day.



```
SELECT
    HOUR(order_time) AS hours, COUNT(order_id) AS order_count
FROM
    orders
GROUP BY HOUR(order_time);
```



Result Grid		
	hours	order_count
▶	11	1231
	12	2520
	13	2455
	14	1472
	15	1468
	16	1920
	17	2336
	18	2399
	19	2009
	20	1642
	21	1198
	22	663
	23	28
	10	8
	9	1



Join relevant tables to find the category-wise distribution of pizzas.



```
SELECT  
    category, COUNT(name) AS count  
FROM  
    pizza_types  
GROUP BY category;
```

Result Grid		
	category	count
▶	Chicken	6
	Classic	8
	Supreme	9
	Veggie	9



Group the orders by date and calculate the average number of pizzas ordered per day.



```
SELECT
    ROUND(AVG(quantity), 0) AS Avg_Pizzas_orderd_per_day
FROM
    (SELECT
        orders.order_date, SUM(order_details.quantity) AS quantity
    FROM
        orders
    JOIN order_details ON orders.order_id = order_details.order_id
    GROUP BY orders.order_date) AS order_quantity;
```

Result Grid		Filter Rows:
	Avg_Pizzas_orderd_per_day	
▶	138	



Determine the top 3 most ordered pizza types based on revenue.



```
SELECT
    pizza_types.name,
    SUM(order_details.quantity * pizzas.price) AS Revenue
FROM
    pizza_types
    JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
    JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.name
ORDER BY revenue DESC
LIMIT 3;
```

Result Grid			Filter Rows:
	name	Revenue	
▶	The Thai Chicken Pizza	43434.25	
	The Barbecue Chicken Pizza	42768	
	The California Chicken Pizza	41409.5	



Calculate the percentage contribution of each pizza type to total revenue.



```
SELECT
  pizza_types.category,
  CONCAT(ROUND(SUM(order_details.quantity * pizzas.price) / (SELECT
    ROUND(SUM(order_details.quantity * pizzas.price),
      2) AS `Total Revenue`
    FROM
      order_details
    JOIN
      pizzas ON pizzas.pizza_id = order_details.pizza_id) * 100,
    2),
    '%') AS revenue
FROM
  pizza_types
  JOIN
  pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
  JOIN
  order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.category
ORDER BY revenue DESC;
```

Result Grid			Filter R
	category	revenue	
▶	Classic	26.91%	
	Supreme	25.46%	
	Chicken	23.96%	
	Veggie	23.68%	



Analyze the cumulative revenue generated over time.



```
select order_date,round(sum(revenue) over (order by order_date),2) as cum_sales
from
(SELECT
    orders.order_date,
    SUM(order_details.quantity * pizzas.price) AS revenue
FROM
    order_details
    JOIN
    pizzas ON order_details.pizza_id = pizzas.pizza_id
    JOIN
    orders ON orders.order_id = order_details.order_id
GROUP BY orders.order_date) as sales;
```



Result Grid			Filter R
	order_date	cum_sales	
▶	2015-01-01	2713.85	
	2015-01-02	5445.75	
	2015-01-03	8108.15	
	2015-01-04	9863.6	
	2015-01-05	11929.55	
	2015-01-06	14358.5	
	2015-01-07	16560.7	
	2015-01-08	19399.05	
	2015-01-09	21526.4	
	2015-01-10	23990.35	
	2015-01-11	25862.65	
	2015-01-12	27781.7	
	2015-01-13	29831.3	



Determine the top 3 most ordered pizza types based on revenue for each pizza category.



```
select name, revenue from
(select category,name,revenue,
rank() over(partition by category order by revenue desc) as rn
from
(SELECT
    pizza_types.category,
    pizza_types.name,
    SUM(order_details.quantity * pizzas.price) AS revenue
FROM
    pizza_types
    JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
    JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.category , pizza_types.name) as sales) as b
where rn<=3;
```

Result Grid   Filter Rows:		
	name	revenue
▶	The Thai Chicken Pizza	43434.25
	The Barbecue Chicken Pizza	42768
	The California Chicken Pizza	41409.5
	The Classic Deluxe Pizza	38180.5
	The Hawaiian Pizza	32273.25
	The Pepperoni Pizza	30161.75
	The Spicy Italian Pizza	34831.25
	The Italian Supreme Pizza	33476.75
	The Sicilian Pizza	30940.5
	The Four Cheese Pizza	32265.700000000065
	The Mexicana Pizza	26780.75
	The Five Cheese Pizza	26066.5





Conclusion



This project successfully demonstrated the use of SQL to analyze pizza sales data and extract meaningful business insights. By applying SQL concepts such as joins, aggregate functions, subqueries, and window functions, key performance metrics including total revenue, customer ordering patterns, popular pizza types, category-wise sales, and revenue trends were identified.

The analysis highlights how SQL can transform raw transactional data into valuable information that supports data-driven decision-making. Overall, this project strengthened my SQL skills and showcased the practical application of database querying techniques in solving real-world business problems.



**THANK
YOU**

